

درس سیستم عامل

تمرین عملی سوم سوال دوم

402105727

متین باقری

402105802

فاطمیما تیمارچی

- فایل های تغییر یافته و عکس خروجی در همین پوشه ضمیمه شده است .

(آ)

با استفاده از دستور make پروژه را کامپایل کردم :

```
Tlmarchi-Bagheri@debian:~/Downloads/zocker-master$ make  
gcc -Wall -Wextra -std=c99 -o zocker main.c
```

(ب)

سوال :

در مسیر `/proc/<PID>/ns/` در کل یک پوشه هست که فایلی که در آن قرار دارند یک `symlink` یا یک لینک نمادین هست که این لینک به `kernel namespace` اشاره میکنه . در لینوکس مسیر `/proc/<PID>/ns/` شامل لینک هایی است که هر کدام اشاره به `namespace` مرتبط با اون پردازنده دارند . به طور خاص `/proc/self/ns/pid` اشاره به `pid namespace` فعلی پردازنده فعلی داره که ما میتونیم با استفاده از دستور `readlink` این مقدار رو بخونیم وقتی من دستور رو در ترمینال زدم خروجی برابر `4026531836` شد که این مقدار شناسه ای یکتا برای فضای `pid namespace` هست که توسط کرنل به این فرایند اختصاص داده شده است . بنابراین اگه دو پردازنده در فضای `pid namespace` مشترکی باشند خروجی این دستور برای این دو یکسان است و تنها در صورتی خروجی فرق میکنه که فضا ها `unshared` باشند . در اینجا پس از اجرای دستور مقدار نشون داده شده `pid namespace` مربوط به فرزند کانتینر هست و در نتیجه فرایند ما در یک `namespace` جدید و جدا از پدر اجرا میشه . پس از زدن دستور `make` و دستور نوشته شده در صورت سوال خروجی به صورت زیر میشه :

```

Timarchi-Bagheri@debian:~/Downloads/zocker-master$ make
gcc -Wall -Wextra -std=c99 -o zocker main.c
Timarchi-Bagheri@debian:~/Downloads/zocker-master$ ./zocker run --name hello 'readlink /proc/self/ns/pid'
Running child with pid: 7297
pid:[4026531836]
[Parent] Stopping...

```

ج

سوال :

روش clone(2) : در این روش با استفاده از syscall ای به نام clone یک فرایند جدید ساخته میشود و همزمان namespace جدیدی هم ایجاد میشه . چون در یک syscall همه چیز انجام میشه پس راحت تر است و همچنین کنترل بسایر دقیقی روی فرزند های یک فرایند داره . اما نیاز به تنظیم دقیق flag ها دارد و همچنین چون مدیریت آن به صورت دستی انجام میشه و مدیریت مستقیم استک و پارامتر های فرزندان بسیار پیچیده است پس مدیریت بسیار دشوار خواهد بود .

روش fork(2)+unshare(2) : در این روش ابتدا با استفاده از unshare یک namespace جدید میسازیم و فرایند فعلی رو از فضای فعلی جدا میکنیم سپس با fork یک فرایند جدی در این namespace جدی میسازیم . این روش ساده تر است چون پیاده سازی و درک آن راحت تر است اما به دلیل آنکه دو syscall مجزا داریم کمی overhead داریم

دلیل آسان بودن روش دوم : علت ساده تر بودن روش fork(2)+unshare(2) این است که پیاده سازی و debug و نگهداری کد در این روش راحت تر است همچنین در این روش ما با توابع آشنا مانند fork و unshare کار میکنیم بنابراین پیاده سازی این روش در زبان های سطح بالا راحت تر است

تصویر خروجی :

ابتدا فایل main.c رو تغییر دادم . در این فایل تنها تابع run_container رو تغییر یافته رو در ادامه گذاشتم . با استفاده از unshare میام فرایند فعلی رو از بقیه namespace های سیستم عامل جدا میکنم . در این حالت تو تا namespace رو جدا میکنیم اول با استفاده از CLONE_NEWPID یک namespace pid جدید میسازیم تا فرایند هایی که بعد از این ساخته میشن مقدار pid آن ها از یک شروع شود . سپس با استفاده از

CLONE_NEWNS میایم یک mount namespace جدید میسازیم که تغییرات در فایل سیستم مثل mount فقط داخل این namespace دیده بشه و روی سیستم اصلی اثری نزاره. پس در کل ما یک فرایند رو ایزوله کردیم. حالا در این تابع با استفاده از دستور perror(fork) میام از یک namespace جدید استفاده میکنم که فرایندم رو ایزوله کنم. به کد والد هم wait اضافه میکنم تا والد تا اتمام زمان اجرای فرزند صبر کند.

کد تابع run_container :

```
int run_container(struct config cfg) {
    pid_t pid;
    if (unshare(CLONE_NEWPID | CLONE_NEWNS) < 0) {
        perror("unshare");
        return 1;
    }

    pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {

        printf("Child PID in new namespace: %d\n", getpid());
        execl("/bin/sh", "sh", "-c", cfg.command, NULL);
        perror("execl");
        exit(1);
    } else {

        waitpid(pid, NULL, 0);
        printf("[Parent] Child finished, PID in parent
namespace: %d\n", pid);
    }

    return 0;
}
```

پس از این تغییرات خروجی به صورت زیر میشود :

```
Timarchi-Bagheri@debian:~/Downloads/zocker-master$ su
Password:
root@debian:/home/fatima-t/Downloads/zocker-master# ./zocker run --name hello 'readlink /proc/self/ns/pid'
Child PID in new namespace: 1
pid:[4026533985]
[Parent] Child finished, PID in parent namespace: 5303
root@debian:/home/fatima-t/Downloads/zocker-master# make
make: 'zocker' is up to date.
```

اگر دستور رو بدون sodu اجرا کنیم اجر نمیشه پس باید ابتدا وارد root بشیم و سپس این دستور رو اجرا کنیم . همان طور که میبینید خروجی متفاوت از بخش قبلی است و الان مقدار pid بچه برابر 1 است و مقدار pid در این قسمت برابر 4026533985 است در حالی که در بخش قبلی برابر 4026531836 بود

(د)

سوال :

برای ایجاد یک namespace جدید مثل CLONE_NEWPID یا PID mount باید قابلیت CAP_SYS_ADMIN رو داشته باشیم برای همین ساختن یک namespace جدید یک عملیات سطح بالای سیستمی هست که نیازمند capability خاصی در لینوکس هست . بنابراین کاربر این capability رو نداره و نمیتونه namespace جدید بسازه یا mount هارو از هم جدا کنه نیاز به دسترسی root داره . بنابراین دستور unshare(CLONE_NEWPID); که در کد من هست یا اجرای zocker run در فضای جدید نیاز به سطح دسترسی root داریم و کاربر به طور پیش فرض نمیتونه اینکار رو انجام بده برای همین اجرا برنامه بدون استفاده از دستور sudo منجر به خطا میشه

تصویر خروجی :

با استفاده از دستور ./zocker sudo setcap cap_sys_admin+ep تقریباً تمام دسترسی های مدیریت سیستمی را مشابه آنچه که کاربر root داره به zocker میده (اجازه تنظیم capability رو به zocker میده) . بنابراین از اونجایی

که unshare تنها با دسترسی root انجام میشود پس از زدن این دستور و دادن دسترسی مشابه root به zocker دیگه میتونیم بدون زدن دستور sudo این رو اجرا کنیم . عکس خروجی به صورت زیر میشه (همان طور در عکس میبیند قبل از زدن دستور setcap دستور ما اجرا نمیشد اما پس از زدن setcap در root و دادن دسترسی های لازم یه zocker , دستور ما اجرا شد):

```
Timarchi-Bagheri@debian:~/Downloads/zocker-master$ ./zocker run --name hello 'readlink /proc/self/ns/pid'
unshare: Operation not permitted
[ERR] Running container failed due to some internal errors.
Timarchi-Bagheri@debian:~/Downloads/zocker-master$ su
Password:
root@debian:/home/fatima-t/Downloads/zocker-master# sudo setcap cap_sys_admin+ep ./zocker
root@debian:/home/fatima-t/Downloads/zocker-master# exit
exit
Timarchi-Bagheri@debian:~/Downloads/zocker-master$ ./zocker run --name hello 'readlink /proc/self/ns/pid'
Child PID in new namespace: 1
pid:[4026534105]
[Parent] Child finished, PID in parent namespace: 6679
Timarchi-Bagheri@debian:~/Downloads/zocker-master$
```

حالا makefile رو به صورت زیر زیر تغییر میدهیم . با اضافه کردن cap_sys_admin+ep به فایل برخی قابلیت های سیستمی (capability) را میدهیم بنابراین دیگر بدون دستور sudo اجرای باینری zocker ممکن است .

```
CC = gcc
CFLAGS = -Wall -Wextra -std=c99
TARGET = zocker
SOURCES = main.c

$(TARGET): $(SOURCES)
    $(CC) $(CFLAGS) -o $(TARGET) $(SOURCES)
    @echo "Setting CAP_SYS_ADMIN capability..."
    sudo setcap cap_sys_admin+ep $(TARGET)

clean:
    rm -f $(TARGET)

.PHONY: clean
```

حال با زدن دستور make بدون اینکه وارد root بشیم دستور اجرا میشه :

```
fatima-t@debian:~/Downloads/zocker-master$ make
gcc -Wall -Wextra -std=c99 -o zocker main.c
Setting CAP_SYS_ADMIN capability...
sudo setcap cap_sys_admin+ep zocker
[sudo] password for fatima-t:
fatima-t@debian:~/Downloads/zocker-master$ ./zocker run --name hello 'readlink /proc/self/ns/pid'
Child PID in new namespace: 1
pid:[4026533863]
[Parent] Child finished, PID in parent namespace: 4047
fatima-t@debian:~/Downloads/zocker-master$
```